The Organisation as the Executable Unit: Towards an Executable Organisational Operating Model for AI-Native Software Engineering

Sudha Manimaran *Independent Researcher*

Version 1.0 · August 2026

Abstract

Software engineering has historically depended on organisational structures, shared objectives, delegated authority, reviewed decisions, and institutional knowledge that outlives any individual contributor, to sustain systems across changes in personnel and technology. Contemporary artificial intelligence systems increasingly perform substantial engineering work, including code generation, architectural proposal, and review, yet operate largely outside these organisational structures: without delegated authority, without persistent organisational responsibility, and without accountability that survives the interaction in which a decision was made. The paper argues that this absence of organisational structure, not insufficient model capability, is the primary obstacle to safely extending AI autonomy across the software engineering lifecycle. It proposes an executable organisational operating model in which the organisation itself is the unit that persists, not the individual participant or the AI agent. Objectives justify capabilities. Capabilities expose governed services. Services are delivered through accountable roles. Roles are fulfilled by interchangeable participants. Those participants may be human, artificial, or external. They produce artefacts, evidence, and organisational knowledge. The paper distinguishes this approach from existing agent-orchestration and workflow-coordination systems, which coordinate interactions and activities but do not execute an organisation with explicit authority, accountability, and institutional memory. It discusses the conditions under which this becomes necessary as the number of autonomous participants in an engineering organisation grows, the limitations of governance as a guarantee of process rather than judgement, and the extension of the model beyond software engineering to other knowledge-intensive domains.

1. Introduction

Software engineering has never been an individual activity. Enduring software systems are produced by organisations, whose capability arises not from employing highly capable individuals but from coordinating specialised responsibilities through governance, accountability, and accumulated knowledge. Authority is granted rather than assumed. Responsibilities attach to organisational roles rather than to the individuals who currently occupy them. Decisions are reviewed before they become organisational commitments. Knowledge is preserved independently of the individuals who created it. These principles were not designed in the abstract; they exist because they let software systems outlive individual engineers, individual projects, and successive generations of technology.

Existing software methodologies were designed around a human workforce and its characteristics. Artificial intelligence changes that premise. Current AI systems automate substantial engineering activity, producing software, documentation, and architectural proposals of real quality, without participating in the organisational structures that make engineering sustainable. They hold no delegated authority, no organisational responsibility, and no accountability that outlives the interaction in which a decision was made. Once that interaction concludes, the reasoning behind the decision typically disappears with it rather than becoming part of an organisation's institutional knowledge.

The paper argues that this is the central obstacle to extending AI autonomy safely across the engineering lifecycle, and that it is an organisational problem rather than a capability problem. The industry's response to date has concentrated almost entirely on increasing what individual AI systems can do: coding assistants evolving into autonomous engineering agents, and a rapidly growing set of orchestration frameworks coordinating increasingly specialised systems (Chen et al., 2024). That focus addresses the participant. It does not address the organisation the participant operates inside, a gap that recent work on accountability in AI-powered software development has begun to document from a different angle, examining how responsibility is currently assigned, largely through vendor terms of service rather than organisational structure (Treude, 2026).

2. Why Coordination Mechanisms Are Not Organisations

Prompts coordinate individual interactions. Workflows coordinate sequences of activity. Agent-orchestration frameworks coordinate computational participants. None of these mechanisms executes an organisation in the sense the term is used here: none represents explicit, persistent authority, accountability, or institutional memory that survives the interaction, the workflow run, or the individual agent that happens to be participating at a given moment. This is consistent with classical coordination theory, which treats coordination as the management of dependencies among activities (Malone & Crowston, 1994); coordinating activities is a narrower problem than governing the organisation those activities occur within, and the two should not be conflated.

This distinction matters because the failure modes of ungoverned AI participation are organisational failure modes, not capability failure modes. A decision made independently by an AI agent, however well-reasoned, carries no accountable chain behind it if nothing external to that agent recorded who or what approved it, on what evidence, and under what authority. Without that record, an organisation cannot confirm that a given failure will not recur, cannot reconstruct why a decision was made once the participant that made it has been replaced, and cannot distinguish a well-justified engineering choice from an unreviewed one after the fact.

Human engineering organisations solve this problem through mechanisms refined over decades: authority that can be withheld as well as granted, work checked by someone other than whoever performed it, and a record that persists independently of any single individual (Finkelstein, 2009). This paper takes the position that these mechanisms should apply to artificial participants for the same reason they apply to human ones, not because artificial intelligence is untrustworthy, but because no participant, human or artificial, should be able to unilaterally certify its own work within an organisation that depends on that certification being reliable. Recent work reframing autonomous agents as organisational actors subject to agency theory and decision-rights frameworks arrives at a related conclusion from corporate-governance theory rather than software engineering practice (Chinnaraju, 2026); this paper takes software engineering practice, rather than governance theory in general, as its starting point.

3. An Executable Organisational Operating Model

The paper proposes modelling the organisation as a persistent structure whose identity does not depend on which participants currently occupy it. Organisations derive continuity not from the people who currently work within them but from structures that persist despite continual changes in personnel: objectives remain, responsibilities remain, standards remain, governance continues, and organisational knowledge accumulates, even as individual participants change. The same principle, this paper argues, should hold when some or all of an organisation's participants are artificial.

The term executable requires precise definition, because executable process already has an established meaning. That established meaning, the sense in which BPMN and workflow engines are executable, differs from what is intended here and could otherwise be mistaken for it. A workflow engine executes a process: it interprets a description of activities and control flow, and enforces the order in which tasks occur. It does not represent, and therefore cannot enforce, the organisational constructs that determine whether a given transition is legitimate independent of sequencing, who holds authority to perform an action, who holds the separate authority to approve it, what evidence a state change requires before it is accepted, or how the outcome of that check feeds back into the standards future transitions are evaluated against. This paper defines an organisational model as executable if its organisational constructs, authority constraints, accountability relationships, service obligations, evidence requirements, and governance rules, are represented in a form that a computational system interprets and enforces during operation, rather than merely documented for human reference and applied at human discretion. On this definition, executability is a property of an organisation's authority and accountability structure, not of its process sequencing; a system can execute a workflow without executing an organisation in this sense, which is the gap Section 2 argues current AI tooling has not closed.

The model represents the organisation as a chain of derived entities. Objectives justify the existence of capabilities: an organisation requires certain abilities because it has certain purposes it must fulfil, not the reverse. Capabilities expose governed services, stable interfaces through which one part of the organisation provides value to another without exposing how that value is internally produced. Services are delivered through accountable roles, which establish sustained organisational responsibility independent of who or what currently fulfils them. Roles are fulfilled by participants, human engineers, AI agents, or external systems, who perform the activities required to realise those

responsibilities and produce artefacts, evidence, and, ultimately, organisational knowledge.

This sequence is significant because it separates concepts that are traditionally conflated in software engineering practice. Objectives are not capabilities. Capabilities are not departments. Services are not activities. Roles are not participants. Participants are not technologies. Each of these is a distinct organisational abstraction with a different rate of change: objectives evolve slowly, capabilities more slowly still, while participants and their underlying technologies may change continuously. Separating these layers allows an organisation to evolve technologically without repeatedly redefining itself organisationally, and allows successive generations of artificial intelligence to be adopted as they mature without requiring the organisation's governance, accountability, or institutional knowledge to be rebuilt each time.

The chain described above is not static; it closes on itself through a learning loop. Every service interaction generates evidence of its own effectiveness. Evidence accumulates into organisational knowledge. Organisational knowledge refines the capability definitions, standards, and governance rules through which future services are delivered, so that improved capabilities produce higher-quality services in turn: capability generates service, service generates evidence, evidence generates knowledge, and knowledge strengthens capability. This closed loop is what makes the model adaptive rather than merely stable.

The loop also makes explicit a distinction that is otherwise easy to elide: organisational learning is not the same as participant learning. Replacing an AI participant with a more capable underlying model improves that participant; it does not, by itself, improve the organisation. Organisational learning occurs only when evidence changes the operating model itself, a revised review standard, a tightened governance rule, an updated capability definition, a change that persists independently of which participant is subsequently assigned to the corresponding role. This distinction matters precisely because the paper has argued that the operating model, not the participant, is the unit that persists; a learning mechanism confined to participants would leave that unit unchanged no matter how much individual performance improved.

4. Authority, Evidence, and the Limits of Self-Certification

Within this model, two mechanisms do the load-bearing work of accountability. First, the authority to perform a piece of work and the authority to approve it are distinct grants; no participant holds both for the same piece of work, regardless of whether that participant is human, artificial, or an external system. Second, completion is a proposal rather than a fact: a defined requirement, a passed test, a completed review, or a recorded decision, must be satisfied and independently checked before work is considered done. Neither mechanism is novel in isolation; both are long-standing practice in mature human engineering organisations. What the paper argues is novel is applying them to artificial participants by default and as a structural property of the organisation, rather than as a policy that depends on a human remembering to enforce it, which is the condition under which most current AI tooling operates.

It is important to state plainly what this buys and what it does not. Structural governance of this kind proves that a process was followed. It does not prove that the underlying judgement was good. An AI participant approving another AI participant's work can clear every gate, evidenced and traceable, and still be wrong. The claim made for this model is auditability, not correctness: it ensures that engineering decisions remain explainable, reviewable, and attributable, not that they are always right. The paper treats this as an important and underexamined distinction for any system that governs autonomous participants.

5. Scale as the Forcing Function

A handful of AI participants can be coordinated adequately through prompts, workflows, and direct human supervision. The paper argues this coordination model does not scale, and that the point at which it breaks is organisationally, not technically, determined. As autonomy increases, a typical engineering organisation is likely to manage not one or two AI systems but many, specialised across requirements, architecture, implementation, verification, security, compliance, and operations. Coordinating that many autonomous participants simultaneously is not solved by improving any individual participant's competence; it requires an organisational abstraction for authority, accountability, and institutional knowledge that operates at that scale. This is consistent with a broader pattern in enterprise computing, in which new coordination infrastructure becomes necessary not when a new technology becomes possible but when an existing coordination mechanism breaks under scale.

6. Generalisation Beyond Software Engineering

The model as described, objectives, capabilities, governed services, accountable roles, evidence, and organisational learning, is not expressed in terms specific to software. Software engineering is, this paper suggests, the first domain in which this problem becomes visible, because it is the first knowledge-intensive discipline in which artificial participants have become capable enough to expose the absence of an organisational model around them. Whether the theory holds with the same rigour in other knowledge-intensive domains, healthcare, financial services, legal services, scientific research, has not been established, and it is not presented here as a settled claim. It is treated as a plausible extension worth testing empirically in future work, not as a conclusion this paper argues for.

7. Limitations

The paper notes three limitations to the position argued here. First, as stated in Section 4, structural governance provides auditability, not a guarantee of correctness; an organisation adopting this model still depends on the underlying competence of its participants and the quality of its evidence standards. Second, the model as described requires an organisation's capabilities, services, and dependency structure to be explicitly authored before engineering work can run through it; this is nontrivial setup work whose cost has not been characterised empirically here. Third, the claims in Sections 5 and 6 concerning scale and cross-domain generality are argued from first principles and from the internal logic of the model; they have not yet been validated against longitudinal data from organisations operating at the scale described, and that validation is treated as necessary future work.

8. Conclusion

This paper has argued that the central obstacle to extending AI autonomy safely across software engineering is organisational rather than technical: artificial participants increasingly perform substantial engineering work without operating inside organisational structures that grant delegated authority, enforce independent review, or preserve institutional knowledge beyond the interaction in which a decision was made. It proposed an executable organisational operating model in which the organisation, rather than any individual participant, is the unit that persists, and in which authority, evidence, and accountability apply to artificial participants by default rather than as optional policy. It distinguished this from prompt-based and workflow-based coordination mechanisms, which do not execute an organisation in this sense, and outlined the conditions, principally scale, under which this distinction is expected to become organisationally necessary rather than merely desirable.

References

Chen, S., Liu, Y., Han, W., Zhang, W., & Liu, T. (2024). A survey on LLM-based multi-agent system: Recent advances and new frontiers in application. *arXiv preprint arXiv:2412.17481*.

Chinnaraju, A. (2026). When AI agents act: Governance, accountability, and strategic risk in autonomous organizations. *International Journal of Research and Scientific Innovation*, 12(12).

Finkelstein, A. (2009). *Software engineering governance: A briefing*. Tutorial presented at the 31st International Conference on Software Engineering (ICSE 2009).

Malone, T. W., & Crowston, K. (1994). The interdisciplinary study of coordination. *ACM Computing Surveys*, 26(1), 87–119. https://doi.org/10.1145/174666.174668

Treude, C. (2026). Accountable agents in software engineering: An analysis of terms of service and a research roadmap. *Proceedings of the 3rd ACM International Conference on AI-Powered Software (AIware 2026)*. arXiv:2605.04532.

Author Biography

Sudha Manimaran is an independent IT consultant and researcher based in Chennai, India. She holds a Master of Engineering in Industrial Engineering from the College of Engineering, Guindy. Prior to establishing her independent consulting practice in 2016, she spent nearly two decades in the IT industry, including leadership roles at Accenture, Fidelity Investments, Wipro, HCL Perot Systems and Tata Consultancy Services, where she led large-scale software delivery, systems integration and organisational transformation initiatives. Her current research focuses on AI-native

software engineering, organisational governance and executable organisational operating models that enable the integration of autonomous AI participants into software engineering organisations.